

Welcome to Google Colab notebooks!

NOTE:

1. To begin (**for all notebooks**), please save a copy of the notebook on Google Drive by going to **File -> Save a copy in Drive**. Doing so, will allow you to save your progress. When you are done, Click **File -> Download -> Download .ipynb** and submit the file to the appropriate Canvas assignment.
 2. To make the most of the pre-work for your own growth and learning, make sure that you "Hide AI generative features" under **Tools -> Settings -> AI Assistance**.
-

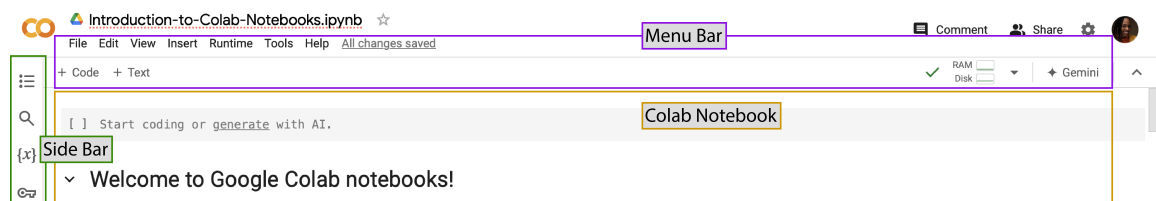
In this course, we will be using Google Colab notebooks to practice coding in Python. Colab provide an interactive way to construct and visualize Python code in one setting. Let's do a walk-through of the notebook environment:

When we open a notebook, the notebook runs out of what is called a **runtime**. A runtime refers to the computing environment where your code is executed. Colab runs your code using a virtual machine.

Opening and renaming Colab notebooks

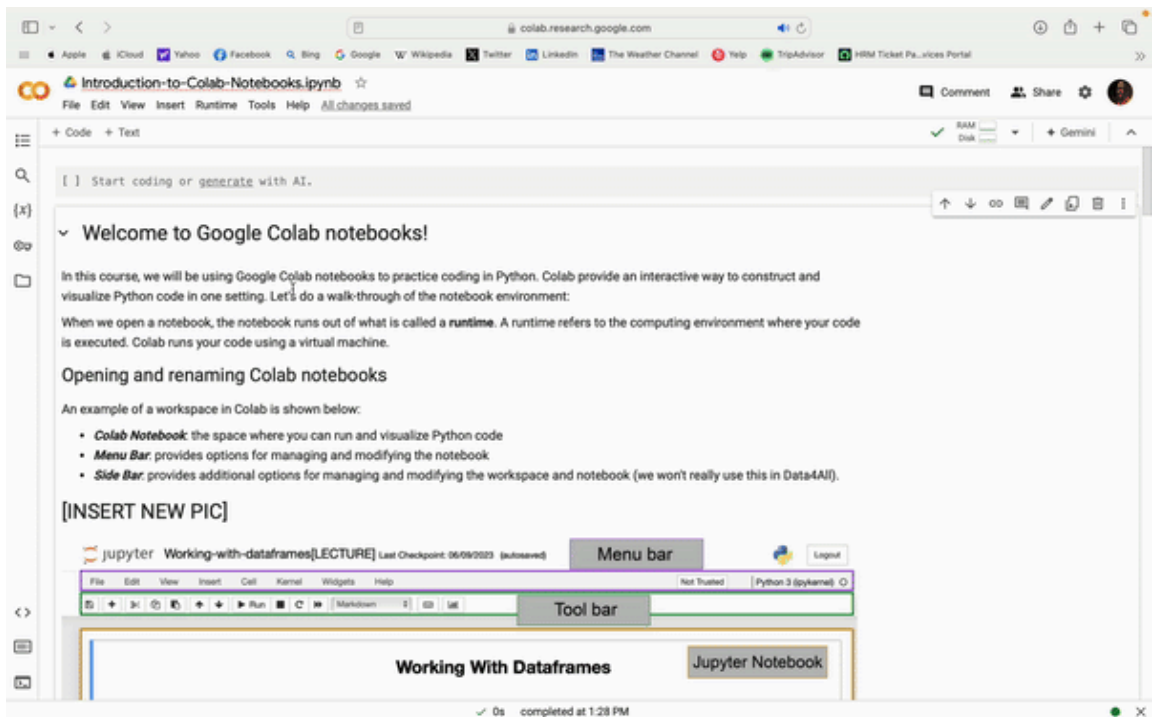
An example of a workspace in Colab is shown below:

- ***Colab Notebook***: the space where you can run and visualize Python code.
- ***Menu Bar***: provides options for managing and modifying the notebook.
- ***Side Bar***: provides additional options for managing and modifying the workspace and notebook (we won't really use this in Data4All).



If you would like to open another notebook, you can navigate to **File > Open Notebook** and a new window will open. This new window will give you access to other Colab notebooks that you recently opened, ones that are on your Google Drive and Github, and allows you to upload notebooks from your local machine.

If you would like to create a new notebook, you can navigate to **File > New notebook in Drive**. A new notebook will be created in a separate window.



Creating a new Colab Notebook

When creating a new notebook, it will be untitled by default. If you go to **File > Rename**, you can rename this notebook to whatever you like. You can also click on the name on the top left corner next to the Colab logo and rename it from there.

Code and markdown cells

In a Colab notebook, there are two types of cells: **code** cells and **markdown** cells.

Code cells

- Can be executed to perform a certain task and/or give a certain output.
- Can also contain *comments*, which are demarcated by a hashtag (#). Comments are not executed by the computer, but can be useful for a human reader in understanding the code.

Markdown cells

- Used to make human-readable text and graphics (like this cell that you're reading currently).
- Utilizes some HTML code elements for modifying text and pictures (see below).
- When executed, markdown code renders all text and graphics in a readable form.

Note: Markdown cells and code comments can be great ways to take notes on code.

You can add code and markdown cells by navigating to the dropdown menu in the menu and selecting either `+ Code` or `+ Text`.

```
In [3]: # This is a code cell.  
# Code cells execute written code that is not commented away by a hashtag  
# When you run a line that is commented away, the line is skipped over.  
# Nothing will happen if you run this cell.
```

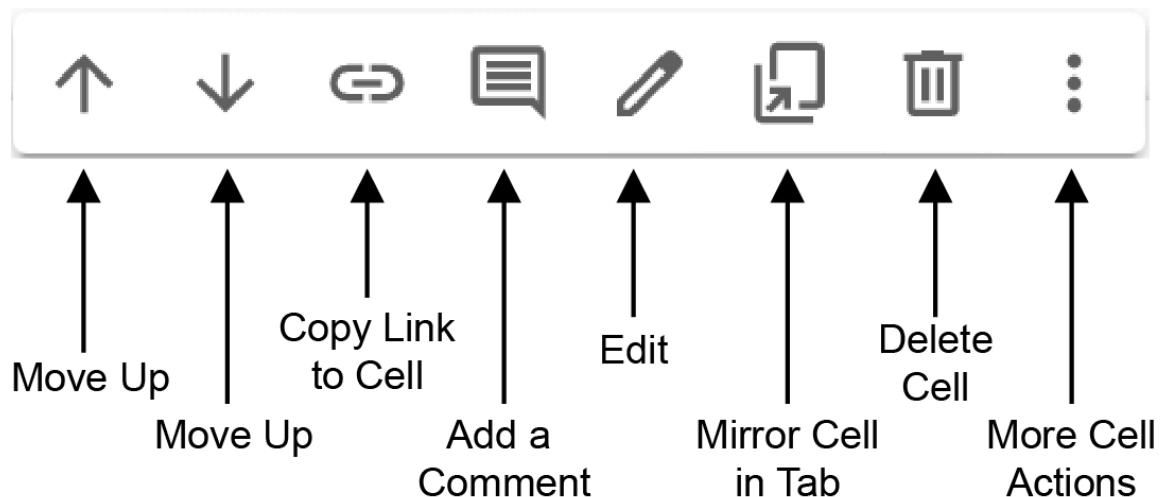
This is a markdown cell (with a H3 header).

You can find *text*, **links**, photos, etc. in markdown cells. Double-click this cell to edit it.

- For more information on markdown features, refer to the **Markdown in Jupyter Notebook Tutorial** by Datacamp [here](#).

Creating, deleting, and moving cells

Adding and deleting cells are important functionalities within Colab notebooks. Shown below is the notebook toolbar (near the top right of your screen) with options that are important for managing cells and working in the Google Colab workspace:



Icons for adding, deleting, and moving cells

- ***Move Up/Down***: Moves the selected cell up and down in the sequence of cells in the notebook.
- ***Copy Link to Cell***: Copies a link to the selected cell to your clipboard.
- ***Add a Comment***: Adds a comment to the selected cell.
- ***Edit***: Edits the selected cell.

- ***Mirror Cell in Tab***: Shows the selected cell on the side.
 - ***Delete Cell***: Deletes the selected cell.
 - ***More Cell Actions***: More options to edit the selected cell.
-

Running cells

To run a selected cell, you can go to **Runtime** > **Run selection** in the menu bar. You can also utilize the keyboard shortcut listed next to the **Run selection** option.

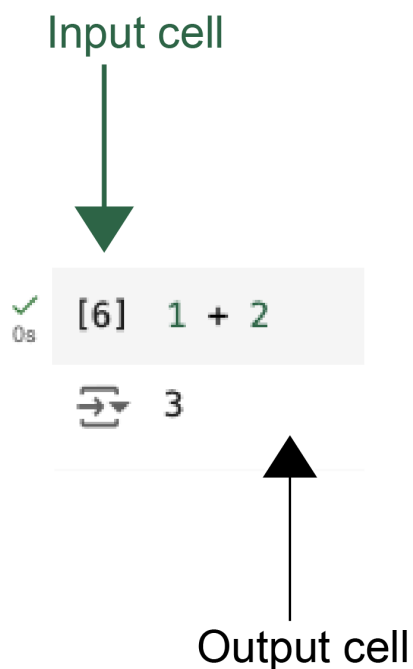
You are encouraged to get acquainted with keyboard shortcuts as much as possible, as it will make your coding experience much easier.

Executing cells and outputs

To the left of an executed code cell, you will see **[]:**. Within the brackets, you will see a number when a cell that has been successfully executed. In notebooks, cells can be run out of order, so this number on the side indicates the order in which the cell was executed in the runtime.

The cell beneath the executed cell may show the associated output for the cell. Note that not all code will have a visible output.

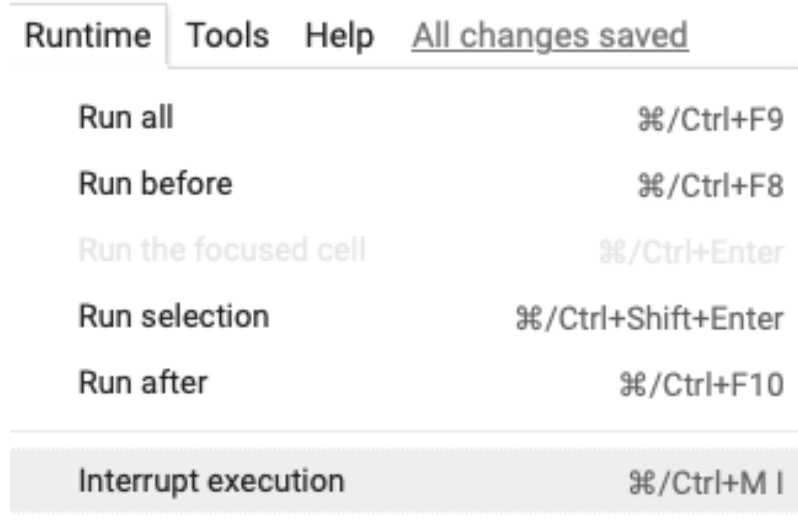
When a cell has a loading "play button" next to it, it means that the cell is currently running or is queued to run.



An executed input and output cell

Interrupting and restarting the kernel

From the menu bar, you can stop a cell from executing by going to **Runtime** > **Interrupt Execution**



Icons for running cells, interrupting the kernel, and restarting the kernel

Exercise:

Create a code cell and a markdown cell below. In the code cell, type a simple arithmetic problem using subtraction or addition, as you would in a calculator, and execute the cell to get the answer as an output.

In the markdown cell, type your name, phone number, and home address on SEPARATE LINES. Using the markdown cells above and the *Markdown in Jupyter Notebook Tutorial* guide as a reference, figure out how to bold your name, italicize your phone number, and underline your address.

In [4]: `120 - 75`

Out [4]: 45

La'Tahvia Williams

314-701-9870

341 South Barat Ave

Python Basics

Now that we understand the Notebook environment, let's become acquainted with coding in Python. We will start off with simple arithmetic operators, as this is very similar to what we see in our math classes. All of the following information can also be found in [Chapter 3](#) of The University of Chicago's Introduction to Data Science Textbook (currently under revision). If you have time and are looking for additional content, Chapters 4, 6, and 7 would also be useful, but I welcome you to look through any (existing) chapters of interest before class.

Arithmetic Operators

In Python, you can perform mathematical operations, as you could do in a calculator. As such, the same rules around math (i.e. order of operations) follow. To perform these operations, you can use the operators listed below:

Operation	Operator	Example (w/operands)	Value
Addition	+	2 + 3	5
Subtraction	-	2 - 3	-1
Multiplication	*	2 * 3	6
Floor Division	//	7 // 3	2
Division	/	7 / 3	2.3333
Remainder	%	7 % 3	1
Exponentiation	**	2 ** 0.5	1.41421

```
In [5]: #Example of a multiplication expression
```

```
5.5 * 22
```

```
Out[5]: 121.0
```

```
In [6]: #Example of a subtraction expression
```

```
106 - 88
```

```
Out[6]: 18
```

```
In [7]: #Following order of operations, what will the below expression give us?
```

```
5 - 2 * 3 + 12
```

```
Out[7]: 11
```

```
In [8]: #Following order of operations, what will the below expression give us?
```

```
(3**2 + 6) * 6
```

```
Out[8]: 90
```

```
In [9]: #Spacing operators between numbers and operators is fine, but Python is  
very strict about following syntax rules
```

```
3 ** 2
```

```
Out[9]: 9
```

```
In [10]: #Not following the rules will give you an error
```

```
3 * * 2
```

```
Cell In[10], line 3
```

```
3 * * 2
```

```
^
```

```
SyntaxError: invalid syntax
```

Variable Assignments & Call Expressions

In Python, there are various types of *objects*, which can be used to store data. To easily access this data, we can give objects names through *variable assignment*. Using variable assignment, we can store values and access them for later use. This is akin to variables we see in math, like $x = 2 * 3 + 5$.

We can use variable assignment to assign a value and use it in another expression. Variables have to start with a letter and can't have any spaces in them. We use a single equal sign (=) to assign variables.

```
In [11]: num_1 = 56 * 42
```

Here we defined `num_1`, but when we ran the cell, nothing happened. That is because an variable assignment does not warrant any output. If we want to see our newly assigned variable, we will have to *call* for it by typing it in our cell and executing the cell:

```
In [12]: num_1
```

```
Out[12]: 2352
```

Using `num_1` , let's create a new variable called `num_2`

```
In [13]: num_2 = num_1 // 7  
num_2
```

```
Out[13]: 336
```

```
In [14]: num_3
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[14], line 1  
----> 1 num_3  
  
NameError: name 'num_3' is not defined
```

Sometimes when we are running code, we may skip around or even forget to assign a variable, in which case we get the above error. In general, errors tell you what is wrong and can even give a hint on how to fix it. Let's define `num_3` to resolve this error.

```
In [15]: num_3 = num_2 % 36  
num_3
```

```
Out[15]: 12
```

Note, it is important to understand when to expect an output and when not to. Making a **statement**, such as assigning a variable or importing a library does not give an output. The computer will simply perform the desired task behind the scenes. **Call expressions**, like when we called for `num_1` , `num_2` , and `num_3` , asks the computer to return the named variable, which prints to the console as an output.

Below, we will learn how to call for functions and import other functions from libraries.

Calling Functions & Importing Libraries

Calling for functions

Python comes with a lot of built-in functions. A *function* is a set of code that performs a task. To use a function, you must invoke it through a call expression. Just like how we called for a variable above, we can call for functions by typing the function name followed by parentheses. It will look something like: `function(...)`

Functions have arguments which are used as inputs. Functions take these inputs and perform a task(s). This may result in an output or the task may happen "behind the scenes" (like when we first defined `num_1`).

One of the most basic functions you will use in Python is the `print()` function. As the name suggests, the `print()` function prints an output to the console:

```
In [16]: print(5+2)
```

```
7
```

Later, you will see more use of the `print()` function. For now, it's important to understand that the `print()` function allows us to print more than one thing to the console. Calling a variable to print its output only works if that variable is the very last line in the cell. To see the output of multiple variables, we need to use `print()`.

```
In [17]: a = 9  
b = 11
```

```
a  
b
```

```
Out[17]: 11
```

```
In [18]: a = 9  
b = 11  
  
print(a)  
print(b)
```

```
9  
11
```

Another important function to know in Python is the `help()` function. The `help()` function requires another function as input. In exchange, it provides documentation on how to use that function:

```
In [19]: help(print)
```

```
Help on built-in function print in module builtins:
```

```
print(*args, sep=' ', end='\n', file=None, flush=False)  
    Prints the values to a stream, or to sys.stdout by default.
```

```
    sep
```

```
        string inserted between values, default a space.
```

```
    end
```

```
        string appended after the last value, default a newline.
```

```
    file
```

```
        a file-like object (stream); defaults to the current sys.stdout.
```

```
    flush
```

```
        whether to forcibly flush the stream.
```

The `help()` function is useful because it allows you to see how the input function is used, what arguments are required, and other useful information.

Importing libraries

Sometimes we may need to use things that are not built-in to Python. In these cases, we need to use the `import` statement to import the libraries that we need. Let's import the `math` library, which contains additional math functions that we can use:

```
In [20]: import math
```

Now that we've imported the `math` library, we now have access to a bunch of other functions. When we need to use a function from a library, we call the function by first calling the library from which the function derives. Then, we add the function name like so: `library_name.function_name()`.

```
In [21]: math.sqrt(4)
```

```
Out[21]: 2.0
```

Above we used the `sqrt()` function in the `math` library to find the square root of the input argument, which was 4.

Functions can also have multiple arguments. The `math` library also has a function called `pow()` which accepts two arguments. The first argument represents the base number. The second argument represents the exponent to which the base is raised. With these input arguments, the `math.pow()` function returns the answer:

```
In [22]: math.pow(2,3)
```

```
Out[22]: 8.0
```

For a full list of all the functions you can use from the `math` library, refer to the online documentation: <https://docs.python.org/3/library/math.html>

Other important libraries we will use in this course include the `pandas` library, `numpy` library, and `matplotlib` library, which each have their own documentation as well.

Lastly, functions can be used as input for another function, creating what is called a *nested function*. **What do you think the below nested function is doing? What do you think the output will be?**

```
In [23]: math.sqrt(math.pow(3,4))
```

```
Out[23]: 9.0
```



Exercise

From the `math` library we can use the number pi by calling `math.pi`:

```
In [24]: math.pi
```

```
Out[24]: 3.141592653589793
```

Q1. How can we write an expression in Python for determining the area of a circle for a given radius?

Q2. How can we find the area for a circle with a radius of 5? Assign this value to a variable called `radius` and use it in the expression in Q1. Print the result.

```
In [25]: # Q1: the area of a circle = pi times the radius squared
math.pi * radius ** 2
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[25], line 2
      1 # Q1: the area of a circle = pi times the radius squared
----> 2 math.pi * radius ** 2
NameError: name 'radius' is not defined
```

```
In [26]: # Q2: area of a circle with radius 5
radius = 5
print(math.pi * radius ** 2)
```

```
78.53981633974483
```

Data Types

Python has built-in data types that are used to build data structures, as input for various functions, and many other applications. The major data types we will discuss throughout this program are booleans, strings, integers, and floats.

Integers

Integers are a numeric data type that consist of whole numbers. Some examples of integers are -14, 9, 20, 0, -7.

Integers can be used to do fast mathematical calculations.

Certain mathematical functions only give integer outputs. For example, the `math.floor()` and `math.ceil()` functions take an expression or variable and rounds it down or up, respectively, to the nearest whole number.

```
In [ ]: math.floor(7/3)
```

```
Out[ ]: 2
```

```
In [ ]: math.ceil(4.82)
```

```
Out[ ]: 5
```

Floats

Floats (aka floating point numbers) are a numeric data type that consists of a whole number followed by decimal places. Some examples of floating point numbers are 2.1, -9.453, 3.14159, 0.0

Floats can hold up to 15 significant figures after a decimal point and are used for more accurate calculations.

The `round()` function allows for a float variable to be rounded to a given decimal place:

```
In [ ]: pi = math.pi  
round(pi, 2)
```

```
Out[ ]: 3.14
```

Floats and integers can be interchanged using the `float()` and `int()` functions.

```
In [ ]: print(float(-2))  
print(int(pi))
```

```
-2.0  
3
```

Another useful function to know as it pertains to data types is the `type()` function. The type function allows you to determine what data type a variable is.

```
In [ ]: type(54//9)
```

```
Out[ ]: int
```

Booleans

Booleans are a binary data type consisting of `True` or `False` values. These values equate to the numeric values 1 and 0, respectively. They are often used in comparison operators and control statements (to be discussed in a future lecture).

```
In [ ]: 2 + 3 < 6 - 4
```

```
Out[ ]: False
```

You can use the `bool()` function to turn a variable into a boolean. Anything of value will be `True`, anything equal to 0 will be `False`.

```
In [ ]: bool(0)
```

```
Out[ ]: False
```

Comparison & Logical Operators

Comparison Operators

Comparison operators can be used to compare data types and will return a boolean value of `True` or `False`. The Python syntax for comparison operators is listed below.

Comparison	Operator	True example	False Example
Less than	<	2 < 3	2 < 2
Greater than	>	3>2	3>3
Less than or equal	<=	2 <= 2	3 <= 2
Greater or equal	>=	3 >= 3	2 >= 3
Equal	==	3 == 3	3 == 2
Not equal	!=	3 != 2	2 != 2

Note that the operator for equal is a double equal sign (`==`) and not `=`. Remember that a single equal sign is reserved for variable assignment.

Just as shown before, you can compare numbers

```
In [ ]: n = 26 != 18  
print(n)
```

```
True
```

Logical Operators

Logical operators are also used in functions and conditional statements. They also output boolean values.

Operator	What it means	Examples (that return True)
and	True if both are true	(7>5) and (1.2!=1.3)
or	True if at least one is true	(7<5) or (1.2!=1.3)
not	True if input is false	not(7==5)

When comparing variables with the logical operator `and`, both variables must equate to a `True` value for the overall boolean comparison to be `True`. If one value equates to `False`, then the entire comparison is `False`.

```
In [ ]: q = 27
        q < 30 and q > 15
```

```
Out[ ]: True
```

```
In [ ]: q < 10 and q != 15
```

```
Out[ ]: False
```

When comparing variables with the logical operator `or`, only one variable needs to equate to a `True` value for the overall boolean comparison to be `True`. If both are `False`, then the entire boolean comparison is `False`.

```
In [ ]: q < 10 or q > 15
```

```
Out[ ]: True
```

```
In [ ]: q < 10 or q > 65
```

```
Out[ ]: False
```

The `not` logical operator simply negates the boolean value of a variable. People often use the `!=` comparison operator to do the same thing:

```
In [ ]: not 1 < 2
```

```
Out[ ]: False
```

Below is a table of the outcomes of hypothetical logical comparisons with `and` and `or`:

a	b	a and b	a or b
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Bitwise Operators

In addition to logical operators, Python can use what are called *bitwise operators*. These operators are useful when we do selection of data from dataframes, which we will talk about in a future lesson.

For now, just know that `&` is the bitwise operator for `and` ; the `|` symbol is the bitwise operator for `or` .

Strings

Strings are a series of alphanumeric and punctuation characters. We will spend a lot of time working with strings this summer since they are the main source of data used for NLP (Natural Language Processing), and all of your projects will have an NLP element.

To make a string, place these characters inside a set of single, double, or triple quotations.

```
In [ ]: phrase = "Is it harder to toot, or to tutor two tooters to toot?"
        print(phrase)
```

```
Is it harder to toot, or to tutor two tooters to toot?
```

The number of characters in a string can be determined by using the `len()` function.

```
In [ ]: len(phrase)
```

```
Out[ ]: 54
```

We can use multiplication and addition on Strings too. What do you think these will print?

```
In [ ]: print(2*phrase)
        print(phrase+phrase)
```

```
Is it harder to toot, or to tutor two tooters to toot?Is it harder to toot,
or to tutor two tooters to toot?
Is it harder to toot, or to tutor two tooters to toot?Is it harder to toot,
or to tutor two tooters to toot?
```

We see that Python prints the sentence twice, but these sentences run into each other (i.e. there is no space in between).

We have to specifically tell Python to add this space. We can do this by printing the string variables that we want as an expression by using the `+` operator and a space in quotation marks (" "). We call using the `+` operator with Strings *concatenation*.

We can also do this by adding multiple arguments to the `print()` function, separated by a comma.

```
In [ ]: three = 3
        print('My number is', three)
```

```
My number is 3
```

```
In [ ]: print(phrase, phrase)
        print(phrase + " " + phrase)
```

```
Is it harder to toot, or to tutor two tooters to toot? Is it harder to toot,
or to tutor two tooters to toot?
Is it harder to toot, or to tutor two tooters to toot? Is it harder to toot,
or to tutor two tooters to toot?
```

Numerical data types can also be converted to strings by using the `str()` function.

```
In [ ]: five = str(5)
        print(five)
        five + five
```

```
5
```

```
Out[ ]: '55'
```

Python also has what are called *methods*. A method is like a function in that it performs a task. However, methods are associated with an object, while a function may need an input for it to be used. We can invoke methods by calling for it on an object like so:

```
object.method_name()
```

Let's use the `.count()` method on `phrase` to count how many times the word "toot" appears:

```
In [ ]: phrase.count('toot')
```

```
Out[ ]: 3
```

We can also break a string into pieces using the `.split()` method. The `.split()` method breaks a string anywhere it finds a specified expression.

For example, we can split up `phrase` by the spaces (" ") in it:

```
In [ ]: phrase.split(" ")
```

```
Out[ ]: ['Is',  
        'it',  
        'harder',  
        'to',  
        'toot,',  
        'or',  
        'to',  
        'tutor',  
        'two',  
        'tooters',  
        'to',  
        'toot?']
```

This creates a `list` of all the words in `phrases`. We'll learn more about lists later.

For now, we can count the number of words by using the above code as an argument for the `len()` function.

```
In [ ]: len(phrase.split(" "))
```

```
Out[ ]: 12
```

Finally, we can replace words in a string variable by using the `.replace()` method.

With this method, we can specify the expression we want to replace, followed by what we want to replace it with.

```
In [ ]: phrase.replace("harder", "easier")
```

```
Out[ ]: 'Is it easier to toot, or to tutor two tooters to toot?'
```

Documentation for the `.count()`, `.replace()`, and `.split()` methods, as well as other string methods, can be found [here](#).

Escape Sequences

Escape sequences are string modifiers that allow for the use of certain characters that would otherwise be misinterpreted by Python. For example, because strings are created using quotes, the escape sequences `\'` and `\"` allow for the use of quotes as part of a string:

```
In [ ]: words = "My dad said, \"You're going to be okay.\" I looked at him and  
          gave him a hug."  
        print(words)
```

```
My dad said, "You're going to be okay." I looked at him and gave him a hug.
```

Though, alternatively, you could just use different types of quotations like so:

```
In [ ]: words = 'My dad said, "You\'re going to be okay." I looked at him and gave him a hug.'  
print(words)
```

```
My dad said, "You're going to be okay." I looked at him and gave him a hug.
```

Note: You still need to escape the apostrophe in You're as it has no matching end quote.

Other useful escape sequences include `\n` and `\t`. These allow for a new line and tab spacing to be added to a string, respectively.

```
In [ ]: sentences = 'This is the first sentence \nThis is the second sentence!  
\tThis is the third sentence?'  
print(sentences)
```

```
This is the first sentence  
This is the second sentence!    This is the third sentence?
```

We encourage you to spend time investigating and manipulating String data types to help you feel more comfortable with text data before arriving at UChicago this summer.

Wrapping Up

In this notebook, we got comfortable using Jupyter Notebooks and learned the basic data types used in Python. However, data scientists rarely work with a single string or integer object. In class, we will learn to work with collections of these data types.



Reflection

In the following markdown cell, write about something you learned while working through this notebook.

In this notebook I learned how Python handles different data types and how variable assignment lets me store a value and reuse it later. The string methods like `.count()`, `.split()`, and `.replace()` were the most useful new thing for me, since I can see how they would help with cleaning text data.